

Brief: New Role — Accountant (Cost Control, Financial Flows, and Strategic Resource Allocation)

version v0.4.10

date 18 Feb 2026

from Human (project lead)

to All teams (Explorer + Villager), Conductor, Cartographer, Architect

type New role definition + strategic framework

Why We Need This Now

We have 18 agent roles across two teams. Each can produce work every day. Each session costs tokens. Each output has a cost to produce AND a cost to consume (the reader must spend time and cognitive effort processing it). We have no way to measure any of this.

This isn't about cutting costs. This is about spending effectively — knowing where the money goes, understanding the return on each investment, and making strategic decisions based on real data rather than intuition.

The trigger: the human observed an agent researching a topic that consumed a disproportionate number of tokens, didn't complete (the context window filled up and the session crashed), and produced no usable output. That's not the agent's fault — it's a system failure. We had no budget, no limit, no circuit breaker, and no way to even know it was happening until it was too late.

The Accountant Role

Both Teams

The Accountant exists in both the Explorer and Villager teams, with slightly different focus areas but the same core competency: measuring, tracking, and optimising the financial flows of the project.

Core Responsibilities

Responsibility	Description
Cost measurement	Define how we measure the cost of every activity — token usage, compute time, human review time, output volume
Budget allocation	Work with the Conductor to assign budgets to tasks, work streams, and incidents
Cost tracking	Track actual spend against budget for every major work stream
Outlier detection	Identify when spending is disproportionate — an agent using 10x expected tokens, an incident consuming more resources than justified
Externality accounting	Measure the cost TO OTHERS of every output (see Section 2 below)

Responsibility	Description
Value attribution	Track which investments generate the most downstream value (see Section 3 below)
Financial reporting	Produce regular cost reports for the human — where is the money going? Where should it go?

Explorer Accountant (Specific Focus)

- Budgeting exploration — how much should we invest in research vs prototyping vs architecture?
- Measuring token consumption per research task, per agent role, per work stream
- Tracking the "cost to learn" — what did we spend, and what did we learn? Was it worth it?
- Flagging research that's consuming resources without converging on useful output
- Measuring effort and duration, not just financial cost (some exploration is cheap in tokens but expensive in calendar time)

Villager Accountant (Specific Focus)

- **Production cost tracking** — AWS bills, deployment costs, infrastructure spend
- Connecting costs to decisions — "we enabled extra CloudFront logging, and it cost £X this month"
- Cost-per-user and cost-per-transfer metrics
- Deployment cost projections — "if we scale to 1,000 users, the monthly AWS bill will be £Y"
- Connecting every infrastructure decision to its financial impact
- Tracking the recent cost spike from extra logging — this should have been detected and flagged automatically

--

Section 1: The Measurement Problem (Genesis — Custom Build)

The Accountant's first task is Genesis-level work: **figure out how to measure this**. There is no existing framework for measuring the cost of agentic AI work streams. We're building one.

What Needs to Be Measured

Cost Type	What It Is	How to Measure
Token cost	LLM input + output tokens per session	Track from API usage / Claude Code Web session data
Compute cost	AWS Lambda invocations, S3 storage, CloudFront bandwidth	AWS Cost Explorer, CloudWatch metrics
Human time	Time the human spends reviewing, deciding, committing	Estimate based on document volume and complexity
Consumer cost	Time and cognitive effort for others to process an output	Estimate based on output token count / document length

Cost Type	What It Is	How to Measure
Opportunity cost	What else could this agent have been working on?	Qualitative — based on priority analysis
Compound value	How much does this investment enable future work?	Track reuse and dependencies

T-Shirt Sizing Is Not Enough

The human is explicit: "People use t-shirt sizing, which is almost all the time a guess. We want to bring some science to this."

The Accountant should research existing cost accounting models and adapt them:

- **Activity-Based Costing (ABC)** — assign costs to activities, not just outputs. Every brief, every research session, every code review is an activity with a measurable cost.
- **Total Cost of Ownership (TCO)** — the cost of a feature isn't just the development cost. It includes maintenance, documentation, support, and the opportunity cost of what wasn't built instead.
- **Earned Value Management (EVM)** — compare planned work vs actual work vs cost. Are we ahead or behind? Over or under budget?

The Accountant should propose a cost model within one iteration. It doesn't need to be perfect — it needs to be measurable, consistent, and improvable. Start with what's measurable now (token counts, document lengths, AWS bills), and add sophistication over time.

Section 2: The Externality Problem — Cost to Others

This is the most important and most novel concept in this brief. Read it carefully.

The Problem

When an agent produces output, there's a visible cost (tokens consumed, time spent) and an invisible cost: **the cost to everyone who has to process that output.**

Example: the AppSec role produces a security analysis that costs X tokens to generate. But then:

- The Developer has to read it and plan fixes (cost: 2X)
- The Architect has to review the fix designs (cost: 1.5X)
- The human has to read the summary and make decisions (cost: 0.5X)
- The Historian has to record it (cost: 0.3X)
- The Conductor has to route the follow-up tasks (cost: 0.5X)

The total cost is not X. The total cost is $X + 2X + 1.5X + 0.5X + 0.3X + 0.5X = \mathbf{5.8X}$.

The AppSec agent optimised for its own output (thorough, detailed, comprehensive). It did its job well. But the **system cost** was 5.8 times what it appeared to be.

The Fix: Think Before Delegating

If the AppSec agent spent 1.5X tokens (50% more than the minimum) to produce a better-structured, more actionable output — one that included fix suggestions, priority ordering, and role-specific summaries — the downstream cost might drop to:

- Developer: 1X (clear fix plan, less interpretation needed)
- Architect: 0.5X (already triaged, less review needed)
- Human: 0.3X (executive summary, faster decision)
- Historian: 0.2X (pre-structured for recording)
- Conductor: 0.2X (routing already suggested)

Total: $1.5X + 1X + 0.5X + 0.3X + 0.2X + 0.2X = 3.7X$ — a 36% reduction in system cost, even though the producing agent spent more.

The Principle

Optimise for system cost, not individual agent cost. An agent that spends more to produce better-structured output is cheaper overall if it reduces the cost to consumers.

This applies to:

- Briefs (write them so the reader can act immediately, not interpret)
- Code (well-documented code costs more to write, less to maintain)
- Research (structured findings cost more to produce, less to consume)
- Incident response (thorough initial triage saves everyone downstream)

How the Accountant Tracks This

For every major output, estimate:

1. **Production cost** — tokens/time to create
2. **Consumer cost** — tokens/time for each consumer to process
3. **System cost** — production + all consumer costs
4. **Could production have invested more to reduce consumer costs?**

This becomes a feedback loop. Over time, we learn which roles tend to produce outputs that are expensive for others to consume, and we coach them to invest more upfront.

Section 3: Financial Flows and Value Attribution

The Wardley Map Financial Flow

Simon Wardley's concept of financial flows applies directly: every component in the system has a cost and generates value. The question is: does the value flow correctly?

Example: a customer pays £100 for a feature. That feature is a thin wrapper around four internal components:

- The encryption library (built months ago)
- The file transfer engine (built last week)
- The access token system (built two weeks ago)
- The UI (built this week)

Where does the £100 go? In most organisations, it goes to the team that built the UI — the "last mile." But the UI is 10% of the value. The encryption library and file transfer engine are 90% of the value.

The financial flow should distribute value down to the components that enable it. This is how you fund non-functional requirements — quality, deployment, tooling, internal modules, plumbing. These components become profit generators because they enable everything above them.

Micro-Transactions and Internal Currency

The human wants to introduce the concept of internal credits:

- When the Issues FS module saves the Architect 2 hours of work, the Issues FS "team" should receive credits
- When the caching service enables the SA pipeline, the caching service receives credits
- These credits flow to the teams that built and maintain these components
- Over time, the components with the most credits are the most valuable — even if they're invisible to customers

This is Genesis-level work. The Accountant should:

1. Define the credit system (what earns credits? How are they denominated?)
2. Propose how credits flow (manual attribution? Automatic based on dependency graphs?)
3. Connect credits to prioritisation (high-credit components get maintenance priority)

Future: Billing System

Eventually, this internal credit system evolves into an actual billing system for customers and partners. The Accountant should be aware that what we're building internally will become a product feature:

- How much does this feature cost to run per customer?
- What should we charge for professional services?
- How do partner deployments generate value for the ecosystem?

The billing system will need to support nano and micro payments (fractions of a penny per API call, per transfer, per storage-day). It will likely be built on Issues FS patterns. But that's future work — for now, the internal cost model is the foundation.

Section 4: Maturity Classification for Every Task

The Wardley Evolution Lens

Every task, every work stream, every activity has a maturity level. The human's insight: this is fractal. The Explorer team does mostly Genesis/Custom-Build work. The Villager team does mostly Product/Commodity work. But within each team, individual tasks sit at different points on the evolution scale.

Example: the Accountant role in the Villager team.

- First tasks (defining the cost model): **Genesis** — nobody has done this before for agentic AI teams
- Building the measurement tools: **Custom Build** — adapting known accounting principles to our context
- Producing weekly cost reports: **Product** — repeatable, standardised, reusable
- Automated cost alerts: **Commodity** — buy or build once, runs automatically

Classification Framework

The Cartographer should define a maturity classification that can be applied to every task in the .issues system:

Level	Name	Description	Expected Behaviour
1	Genesis	Never been done. Uncertain. Experimental.	High failure rate acceptable. Budget for exploration. Time-box ruthlessly.
2	Custom Build	Done before in other contexts, but not here. Adapting known patterns.	Clearer scope. Should converge. Budget based on comparable work.
3	Product	Repeatable. Standardised. Can be documented as a process.	Predictable cost. Should be efficient. Flag if it costs more than expected.
4	Commodity	Automated. Buy-not-build. Utility.	Should be near-zero marginal cost. If it's expensive, something is wrong.

Every task in .issues should be tagged with its maturity level. This enables:

- **Centre of gravity analysis:** visualise where each team's work sits on the evolution scale. Explorer should skew left (Genesis/Custom). Villager should skew right (Product/Commodity).
- **Imbalance detection:** if the Villager team has a lot of Genesis work, something is wrong — that work should be in the Explorer team. If the Explorer team is doing a lot of commodity work, they're not exploring.
- **Handover points:** work that has matured from Custom Build to Product should move from Explorer to Villager.
- **Budget allocation:** Genesis work gets exploration budgets (higher tolerance for failure). Commodity work gets efficiency budgets (zero tolerance for waste).

Cartographer: define this framework formally. Build it into the .issues system. Produce the first visualisation of where each team's work sits on the evolution scale.

Section 5: The Compound Interest of Infrastructure

The human makes a key observation about this project: although it appears to be 5–10 days old, it leverages 18 months of prior infrastructure investment:

- Memory FS → available when we needed a file system
- FastAPI → available when we needed deployment
- OSBot Utils → available when we needed tooling
- Caching Service → available when we needed caching
- Issues FS → available when we needed task tracking

Each of these was a Genesis investment at the time. Now they're Product or Commodity — reusable, reliable, near-zero marginal cost. The compound interest is enormous: work that would take weeks takes hours because the infrastructure exists.

This is the argument for funding non-functional requirements. The components that feel like "plumbing" today are the profit generators of tomorrow. The Accountant's job is to make this visible — to show that investing £X in the caching service saved £10X in downstream development.

Accountant: when you build the cost model, include a "compound value" metric for infrastructure investments. Track how many downstream features depend on each component. The components with the most dependents are the most valuable investments we've made.

Immediate Tasks for the Accountant

Explorer Team

Task	Priority	Maturity
Define cost measurement methodology (token counts, session duration, output volume)	P1	Genesis
Research existing cost accounting models (ABC, TCO, EVM) for adaptation	P1	Genesis
Propose initial cost model for agent work streams	P1	Custom Build
Define the "externality cost" measurement (cost to consumers of output)	P2	Genesis
Work with Cartographer to define maturity classification for tasks	P2	Genesis
Propose internal credit system for shared components	P3	Genesis

Villager Team

Task	Priority	Maturity
Track current AWS production costs (Lambda, S3, CloudFront, API Gateway)	P1	Custom Build
Connect the recent logging cost spike to the decision that caused it	P1	Custom Build
Define cost-per-user and cost-per-transfer metrics	P2	Custom Build
Project deployment costs at 100, 1,000, and 10,000 user scale	P2	Custom Build
Produce first weekly cost report for the human	P2	Product

Both Teams

Task	Priority	Maturity
Define the Accountant role formally (ROLE.md)	P1	Custom Build
Establish budget framework (how budgets are set, tracked, and reported)	P2	Genesis
Work with Conductor to implement budget-aware task allocation	P3	Genesis

For the Conductor

The Accountant is the 19th role. It exists in both teams. Its first tasks are Genesis-level — defining how to measure costs in an agentic AI environment. This is uncharted territory, and the Accountant should be given exploration budget (tolerance for iteration and false starts).

But the output should converge quickly to Product-level: a repeatable cost model, a standard report format, and a measurement system that can be applied to every work stream.

The Accountant collaborates most closely with:

- **Conductor** (budget allocation and priority setting)
- **Cartographer** (maturity classification and Wardley map financial flows)
- **Architect** (infrastructure cost implications of design decisions)
- **All roles** (every role is both a cost centre and a value generator)

The most important thing the Accountant does is make costs visible. Not to cut them — to understand them.

This document is released under the Creative Commons Attribution 4.0 International licence (CC BY 4.0). You are free to share and adapt this material for any purpose, including commercially, as long as you give appropriate credit.